

SNo	ObservationDate	State	Confirmed	Deaths	Recovered
10	01/22/2020	Hebei	1	0	0
48	01/23/2020	Hebei	1	1	0
112	01/24/2020	Hebei	2	1	0
147	01/25/2020	Hebei	8	1	0
191	01/26/2020	Hebei	13	1	0
238	01/27/2020	Hebei	18	1	0
287	01/28/2020	Hebei	33	1	0
337	01/29/2020	Hebei	48	1	0
390	01/30/2020	Hebei	65	1	0
449	01/31/2020	Hebei	82	1	0
510	02-01-20	Hebei	96	1	0
578	02-02-20	Hebei	104	1	3
646	02-03-20	Hebei	113	1	3
713	02-04-20	Hebei	126	1	4
783	02-05-20	Hebei	135	1	6
854	02-06-20	Hebei	157	1	13
925	02-07-20	Hebei	172	1	22
995	02-08-20	Hebei	195	1	30
1068	02-09-20	Hebei	206	2	34
1139	02-10-20	Hebei	218	2	41
1211	02-11-20	Hebei	239	2	48
1284	02-12-20	Hebei	251	2	54
1357	02/13/2020	Hebei	265	3	68
1430	02/14/2020	Hebei	283	3	87
1505	02/15/2020	Hebei	291	3	101
1581	02/16/2020	Hebei	300	3	105
1656	02/17/2020	Hebei	301	3	122
1731	02/18/2020	Hebei	306	4	136
1806	02/19/2020	Hebei	306	4	152

Figure 2: Grouping data set of COVID-19 on 'State'

- [facilitating-access-to-covid-19-research/](#)
- [sirm.org/category/senza-categoria/covid-19/](#)
- [dev.to/anujgupta/google-s-25-million-datasets-a-perfect-gift-for-aspiring-data-scientists-3ekh](#)
- [github.com/CSSEGISandData/COVID-19](#)
- [kaggle.com/allen-institute-for-ai/CORD-19-research-challenge/kernels](#)
- [data.humdata.org/dataset/novel-coronavirus-2019-ncov-cases](#)
- [github.com/ieee8023/covid-chestxray-dataset](#)
- [github.com/CSSEGISandData/COVID-19](#)
- [github.com/ieee8023/covid-chestxray-dataset](#)
- [kaggle.com/kimjihoo/coronavirusdataset](#)
- [open-source-covid-19.weileizeng.com/](#)
- [github.com/beoutbreakprepared/nCoV2019/tree/master/latest_data](#)
- [kaggle.com/einsteindata4u/](#)

[covid19/version/4](#)

- [cebm.net/covid-19/covid-19-signs-and-symptoms-tracker/](#)
- [tableau.com/covid-19-coronavirus-data-resources](#)

For the upcoming data analysis tasks, the COVID data set in the comma separated values (CSV) format is downloaded and then evaluated using Python Pandas. These data sets can be directly imported into the DataFrames using Python Pandas. DataFrames are used for high-performance operations on the data sets.

Grouping and aggregation of data on the COVID data set

In the actual data set shown in Figure 1, there are thousands of records with date-wise entries.

First of all, the data set is imported in the DataFrame, which is available in Python Pandas. The following is very small code snippet of Python Pandas that is executed to perform the grouping of data from the COVID data set after importing the CSV data set in

	A	B	C	D
1	Age	Weight	Died	Gender
2	19	90	0	m
3	35	80	0	m
4	26	72	0	m
5	27	75	0	f
6	19	70	1	f
7	23	84	0	f
8	27	88	0	f
9	32	84	1	m
10	42	75	0	m
11	35	82	0	f
12	26	84	0	m
13	26	85	0	m

Figure 3: COVID-19 data set with multiple attributes

DataFrame:

```
import pandas as pd
dataset = pd.read_csv("covid_19_data.csv")
dval = dataset.values
print (dataset)
grouped=dataset.groupby('State')
print(grouped.get_group('Hebei'))
grouped.get_group('Hebei').to_csv('n.csv')
```

With the execution of code, the grouping is done by the *State* and then presented in an understandable and grouped format, as shown in Figure 2. With this approach, the date-wise grouping is performed so that the large data set can be confined and be presented effectively.

Melting of large data sets using Pandas

Another task that is being performed relates to melting or decomposing the huge data set. In the data set shown in Figure 3, the age, gender and death (Yes / No) is recorded.

The data set from other formats can also be imported in DataFrame. Once the data is imported to a DataFrame, then different operations can be done. The following is a code snippet of data melting on the COVID data set:

```
import pandas as pd
dataset = pd.read_csv("covid.csv")
```